

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**

Department of Computer Science and Engineering

**ASSESSMENT TOOL 1 – EXPERIMENTS**

|                         |                                                                                                          |                      |                                   |
|-------------------------|----------------------------------------------------------------------------------------------------------|----------------------|-----------------------------------|
| <b>Course Code:</b>     | CSA06                                                                                                    | <b>Course Title:</b> | Design and Analysis of Algorithms |
| <b>CO Assessed:</b>     | <b>CO3: Articulation</b><br>(BL4, BL5)                                                                   | <b>Assessment:</b>   | Assessment Tool 1 – Experiments   |
|                         |                                                                                                          | <b>Total Marks:</b>  | 100                               |
| <b>CO3 Statement:</b>   | Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication |                      |                                   |
| <b>Student Name:</b>    | M.S.Dinesh                                                                                               | <b>Reg. No.:</b>     | 192424016                         |
| <b>Section / Batch:</b> | I                                                                                                        | <b>Date:</b>         | 04 08 2026                        |

**Instructions to Students**

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

| <b>Question Type</b> | <b>Focus Area</b>                                                                                                   | <b>Questions</b> |
|----------------------|---------------------------------------------------------------------------------------------------------------------|------------------|
| Experiments          | Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication | Q1               |

**SECTION A – Experiments****Code of Conduct**

I (M.S.Dinesh / 192424016) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: \_\_\_\_\_ M.S.Dinesh

## RUBRICS FOR CALCULATION

| Experiments               |           |                                                                                               |                                                             |                                                     |                                                   |
|---------------------------|-----------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------|---------------------------------------------------|
| Criteria                  | Weightage | Level 4 – Exemplary                                                                           | Level 3 – Proficient                                        | Level 2 – Developing                                | Level 1 – Beginning                               |
| Understanding of Concepts | 25%       | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding ; unable to integrate concepts |
| Experimental Design       | 30%       | Well-planned, systematic implementation with optimal solution                                 | Correct implementation with minor issues                    | Basic implementation with errors or inefficiencies  | Incorrect or incomplete implementation            |
| Use of Tools              | 20%       | Effective and advanced use of tools/technologies with proper configuration                    | Appropriate use of tools with correct execution             | Limited or inconsistent use of tools                | Incorrect or minimal use of tools                 |
| Analysis of Results       | 15%       | Insightful analysis with accurate interpretation and validation of results                    | Good analysis with reasonable interpretation                | Basic analysis with limited insights                | Poor or incorrect analysis of results             |
| Documentation             | 10%       | Clear, well-structured documentation with diagrams, code, and results                         | Organized documentation with necessary details              | Basic documentation with missing details            | Poor or incomplete documentation                  |

## Marks Evaluation:

| Criteria                  | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---------------------------|-----|---------------------|----------------------|----------------------|---------------------|
| Understanding of Concepts | 25% |                     |                      |                      |                     |
| Experimental Design       | 30% |                     |                      |                      |                     |
| Use of Tools              | 20% |                     |                      |                      |                     |
| Analysis of Results       | 15% |                     |                      |                      |                     |
| Documentation             | 10% |                     |                      |                      |                     |

|                           |  |
|---------------------------|--|
| <b>Total Marks (100):</b> |  |
|---------------------------|--|

| <b>Faculty In-charge</b> | <b>Course Coordinator</b> | <b>Head of Department</b> |
|--------------------------|---------------------------|---------------------------|
| Name:                    | Name:                     | Name:                     |
| Signature: _____         | Signature: _____          | Signature: _____          |
| Date: _____              | Date: _____               | Date: _____               |

Language: **Python** C++ Java

## QUESTIONS

**Q1**  
Quick Sort  
100 marks

Q1 answered

**Q1** Quick Sort

100 marks

Evaluate the performance of Quick Sort using median-of-three pivot selection on large datasets. Compare execution time with first and last element pivot strategies. Record and plot the results for different input distributions. Interpret how median selection reduces the likelihood of worst-case scenarios and justify its effectiveness.

## Your Code

```
1 import random
2 import time
3 import sys
4
5 # sys.setrecursionlimit(10000)
6
7
8 # ----- pivot Strategies -----
9
10 def quick_sort_first(arr):
11     if len(arr) <= 1:
12         return arr
13     pivot = arr[0]
14     left = [x for x in arr[1:] if x < pivot]
15     right = [x for x in arr[1:] if x >= pivot]
16     return quick_sort_first(left) + [pivot] + quick_sort_first(right)
17
18
19 def quick_sort_last(arr):
20     if len(arr) <= 1:
21         return arr
22     pivot = arr[-1]
23     left = [x for x in arr[:-1] if x < pivot]
24     right = [x for x in arr[:-1] if x >= pivot]
25     return quick_sort_last(left) + [pivot] + quick_sort_last(right)
26
27
28 def quick_sort_median(arr):
29     if len(arr) <= 1:
30         return arr
31     pivot = arr[(arr[0] + arr[-1] + arr[(len(arr) // 2)]) // 3]
32     left = [x for x in arr if x < pivot]
33     middle = [x for x in arr if x == pivot]
34     right = [x for x in arr if x > pivot]
35     return quick_sort_median(left) + middle + quick_sort_median(right)
36
37
38 # ----- timing helpers -----
39
40 def measure_time(func, data):
41     start = time.time()
42     result = func(data.copy())
43     return time.time() - start
44
45
46 # ----- experiment -----
47
48 def run_experiment(size):
49     print(f"Size: {size}")
50
51     random_data = [random.randint(0, size) for _ in range(size)]
52
53     # Use small size for sorted/reverse data with first/last pivot
54     # to avoid recursion errors (they cause worst-case O(n) depth)
55     small_size = min(size, 500)
56     sorted_data = list(range(small_size))
57     reverse_data = list(range(small_size, 0, -1))
58
59     print("Random data:")
60     print("  first pivot :", round(measure_time(quick_sort_first, random_data), 4))
61     print("  last pivot  :", round(measure_time(quick_sort_last, random_data), 4))
62     print("  median-of-3 :", round(measure_time(quick_sort_median, random_data), 4))
63
64     print(f"Sorted data [n={small_size}]:")
65     print("  first pivot :", round(measure_time(quick_sort_first, sorted_data), 4))
66     print("  last pivot  :", round(measure_time(quick_sort_last, sorted_data), 4))
67     print("  median-of-3 :", round(measure_time(quick_sort_median, sorted_data), 4))
68
69     print(f"Reverse sorted data [n={small_size}]:")
70     print("  first pivot :", round(measure_time(quick_sort_first, reverse_data), 4))
71     print("  last pivot  :", round(measure_time(quick_sort_last, reverse_data), 4))
72     print("  median-of-3 :", round(measure_time(quick_sort_median, reverse_data), 4))
73
74
75 # ----- main -----
76
77 if __name__ == "__main__":
78     for size in [100, 500, 1000]:
79         run_experiment(size)
```

## Input

-1

## Output

```
Reverse Sorted data [n=500]:
first pivot : 0.0
last pivot  : 0.01561
median-of-3 : 0.0
```